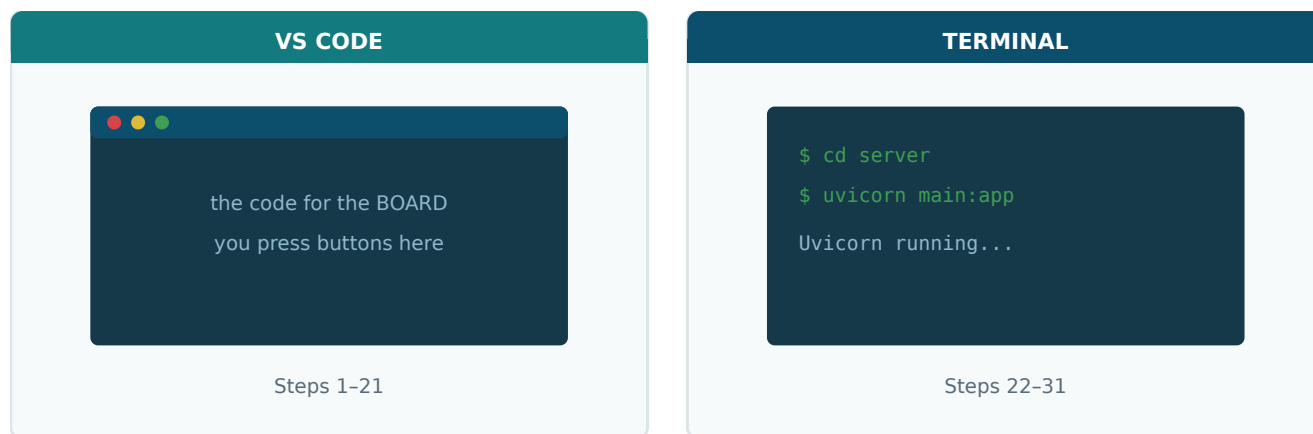


A to B

38 steps. Nothing installed, nothing assumed.

What you're about to do. Install two free programs. Put the code on your laptop. Plug the board in with a USB cable. Send a test program to the board. Read what it says. Then start the AI part and watch the whole thing work.

THE TWO WINDOWS YOU'LL USE



Steps	What happens
1-6	Install VS Code and PlatformIO
7-10	Get the project onto your laptop
11-14	Plug the board in and prove the laptop sees it
15-21	Send the self-test to the board and read the results
22-31	Start the AI server on your laptop
32-38	Connect the two and watch it work

"Terminal" is just a window where you type commands instead of clicking. Step 22 shows you how to open one. Nothing you type there can break your laptop.

PART 1 — Install the software

Steps 1–10. About 20 minutes, mostly waiting.

- 1

Go to `code.visualstudio.com` and download VS Code. ☐

It's free. It's a text editor that programmers use.
You should see: a download starts.
- 2

Install it and open it. ☐

You should see: a mostly-empty dark window with icons down the left side.
- 3

Click the Extensions icon in the left bar — four small squares, one detached. ☐

Fourth or fifth icon down. Or press `Ctrl+Shift+X` (`Cmd+Shift+X` on Mac).
You should see: a search box appear at the top left.
- 4

Type `PlatformIO IDE` into that search box. ☐

You should see: "PlatformIO IDE" as the top result, with an alien-head logo.
- 5

Click Install on it. Then wait. ☐

This one takes 5–10 minutes and downloads a lot. Leave it alone.
You should see: eventually, an alien-head icon appears in the left bar.
What is it? The thing that turns code into something the board understands, and sends it over USB.
- 6

Restart VS Code when it asks. ☐

You should see: a small toolbar appear along the very bottom edge — a tick, an arrow, a plug.
- 7

Get the project folder onto your laptop. ☐

If it's already there, skip to step 9. If not, download it from GitHub as a ZIP and unzip it somewhere you'll find again — Desktop is fine.
You should see: a folder containing `firmware`, `server`, and `hardware`.
- 8

Note where that folder is. You'll need the path twice. ☐

e.g. `/Users/you/Desktop/HardwareHackWADSIH` or `C:\Users\you\Desktop\HardwareHackWADSIH`
You should see: a path you could type out.
- 9

In VS Code: File → Open Folder. Choose the `firmware` folder. ☐

The `firmware` folder itself — not the folder above it. PlatformIO looks for `platformio.ini` in whatever you open, and that file lives in `firmware`.
You should see: `platformio.ini` and a `src` folder in the left panel.
Opened the wrong one? File → Open Folder again. No harm done.
- 10

Wait for the bottom bar to stop saying it's configuring. ☐

First open downloads the ESP32 toolchain. Several minutes. Once only.
You should see: the bottom toolbar settles, with a tick ✓ and a right-arrow → on the left.

PART 2 — Plug in and flash the self-test

Steps 11-21.

- 11

Plug the USB-C cable into the board, and the other end into your laptop. ☐

You should see: a tiny LED on the board light up.
No light? Try a different cable. Many USB cables only carry power, not data — those will never work here even though the light might come on.

- 12

Click the alien-head icon in the left bar, then "Devices". ☐

Under PIO Home. It lists everything plugged into your USB ports.
You should see: an entry like `/dev/cu.usbmodem...` (Mac) or `COM5` (Windows).
Nothing listed? Unplug, replug, refresh. Still nothing → it's the cable, 95% of the time.

- 13

In the left file panel, open the `src` folder. ☐

You should see: `main.cpp`, `selftest.cpp`, `pins.h`, `secrets.h.example` and others.

- 14

Right-click `secrets.h.example` → Copy. Right-click empty space → Paste. Rename the copy to exactly `secrets.h`. ☐

Right-click the new file → Rename. Delete the `.example` from the end.
You should see: both `secrets.h.example` and `secrets.h` in `src`.
Why: the code refuses to build without `secrets.h`. It's separate so your WiFi password never gets uploaded to GitHub.

- 15

Click `secrets.h` to open it. Fill in your WiFi name and password. ☐

Keep the quote marks. Leave `SERVER_HOST` alone for now — step 33.
You should see: your real network name between the quotes. Save with `Ctrl+S` / `Cmd+S`.
Important: the board only works on **2.4GHz** WiFi. If your network is 5GHz-only it will never connect. Most home routers broadcast both.

- 16

At the very bottom of the VS Code window, click where it says `Default` or shows an environment name. ☐

Bottom status bar, near the alien head. This is where you choose WHICH program to send.
You should see: a menu listing `seed_xiao_esp32s3` and `selftest`.

PART 2 continued — send the test to the board

Steps 17–21.

- 17

Choose `selftest` .

☐
- This is the diagnostic program, not the real firmware. It checks every part and reports back.

You should see: `env:selftest` in the bottom bar.
- 18

Click the right-arrow → in the bottom toolbar. That's Upload.

☐
- It builds the code and sends it to the board in one go. First time takes a few minutes.

You should see: a wall of text, ending in green `SUCCESS` .

Red errors? Copy the last 20 lines and send them to me. Don't start editing code.
- 19

Upload failed saying it can't find the port? Hold BOOT, tap RESET, release BOOT, click Upload again.

☐
- Two tiny buttons on the board. This forces it into receive mode.

You should see: the upload proceed.
- 20

Click the plug icon  in the bottom toolbar. That's the Serial Monitor.

☐
- It shows messages the board sends back to your laptop.

You should see: a new panel open at the bottom.
- 21

Press the RESET button on the board and read the results.

☐
- The test runs immediately on boot, so you may need to reset to catch it from the start.

You should see: a list of `[PASS]` and `[FAIL]` lines, then a summary. **It will ask you to press your button and will play a tone.**

Fix the FIRST failure first — the ones below it are often just knock-on effects.

PART 3 — Start the AI server

Steps 22–31. This runs on your laptop, not the board.

22 Open a Terminal. ☐

Mac: press `Cmd+Space`, type `Terminal`, press Enter.

Windows: press the Start key, type `PowerShell`, press Enter.

You should see: a window with a blinking cursor waiting for typing.

23 Check Python is installed. Type this and press Enter: ☐

```
python3 --version
```

You should see: something like `Python 3.11.5`.

"command not found"? Install it from `python.org`, then close and reopen the Terminal. On Windows try `python --version` instead — and use `python` everywhere below.

24 Move the Terminal into the `server` folder. Type `cd` (with a space), then DRAG the `server` folder from your file browser onto the Terminal window, then press Enter. ☐

`cd` means "change directory". Dragging pastes the path so you don't have to type it.

You should see: the prompt now mentions `server`.

25 Confirm you're in the right place: ☐

```
ls
```

Windows PowerShell: `dir`.

You should see: `main.py`, `vision.py`, `requirements.txt` and friends.

Don't see those? You're in the wrong folder. Redo step 24.

26 Install what the server needs: ☐

```
pip install -r requirements.txt
```

You should see: lots of downloading, ending without a red error.

pip not found? Try `pip3` or `python3 -m pip`.

PART 3 continued — API keys and go

Steps 27–31.

27 In VS Code, make a new file in the `server` folder called exactly `.env` ☐

Yes, it starts with a dot and has no name before it. That's correct.

You should see: an empty `.env` file open.

28 Put your API keys in it, one per line, and save. ☐

```
ANTHROPIC_API_KEY=sk-ant-...  
OPENAI_API_KEY=sk-...
```

You should see: two lines with your real keys. No quote marks, no spaces around the `=`.

This file is gitignored, so your keys never leave your laptop.

29 Back in the Terminal, start the server: ☐

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

You should see: Uvicorn running on `http://0.0.0.0:8000`

Leave this window open and running. Closing it stops the server. Open a **SECOND** Terminal for anything else.

30 Open a browser and go to `http://localhost:8000/health` ☐

You should see: a short line of text or JSON — not an error page.

31 Test the AI with no hardware at all. Photograph a medicine box with your phone, put the photo in the `server` folder as `box.jpg`, then in a **SECOND** Terminal (also `cd 'd` to `server`): ☐

```
curl -X POST http://localhost:8000/read_label -F "image=@box.jpg"
```

You should see: the AI describe what's on the box.

This is the most important test in the project. It proves the whole AI half works with zero wiring involved.

PART 4 — Connect the two halves

Steps 32–38. The finish line.

32 Find your laptop's address on your WiFi network. In a Terminal:



```
Mac:      ipconfig getifaddr en0
Windows:  ipconfig
Linux:    hostname -I
```

On Windows, look for the line saying `IPv4 Address`.

You should see: four numbers like `192.168.1.42`. Write it down.

Not `127.0.0.1` — that means "this same computer", and the board isn't this same computer.

33 In VS Code, open `src/secrets.h` and put that address in as `SERVER_HOST`. Save.



You should see: `#define SERVER_HOST "192.168.1.42"` with your number.

34 With `selftest` still selected, click Upload again, then open the Serial Monitor.



You should see: the test run again — and this time `[PASS] Server reachable`.

Still failing? The laptop and the board must be on the **same WiFi network**. Not one on guest WiFi and the other on the main one.

35 Only when every self-test line passes: switch the environment back to `seeed_xiao_esp32s3`.



Same bottom-bar menu as step 16.

You should see: `env:seeed_xiao_esp32s3` in the bottom bar.

Don't skip ahead to this. The real firmware assumes the hardware works — debugging it is far harder than debugging the self-test.

PART 4 continued — the finish line

Steps 36–38.

- 36** Click Upload. Then open the Serial Monitor. ☐
- You should see:** `WiFi connected`, then `Glasses ready`.

- 37** Point the camera at a medicine box and press your button. ☐
- You should see:** the Serial Monitor say it triggered, the server Terminal log a request, and **the speaker read the label out loud.**
- That's the whole product working.**

- 38** Do it ten more times with different objects. ☐
- A tin, something handwritten, something in bad light, something with a barcode.
- You should see:** ten out of ten trigger and speak. Anything less is what you fix next.

Keep these three windows open whenever you're working: VS Code with the Serial Monitor, the Terminal running the server, and a second Terminal for everything else. Almost every problem shows up in one of the first two.